

Plateforme d'Actuariat Assurance

Document technique

École Supérieure Polytechnique de Dakar — Master CCA
Projet n°46 — Année universitaire 2025–2026

Version 1.0 — Août 2026

Sommaire

1. Contexte et objectifs
2. Architecture technique
3. Structure du projet
4. Modèle de données
5. Sécurité
6. Fonctionnalités clés
7. Dépendances externes
8. Installation et déploiement
9. Conventions de code
10. Pistes d'évolution

1. Contexte et objectifs

Ce document décrit l'architecture technique de la Plateforme d'Actuariat Assurance, développée dans le cadre du Projet n°46 du Master CCA de l'École Supérieure Polytechnique de Dakar. Il s'adresse à toute personne souhaitant comprendre, maintenir ou faire évoluer le code source.

L'application gère un portefeuille de polices d'assurance et leurs sinistres associés, avec trois niveaux d'accès utilisateurs, un tableau de bord d'indicateurs actuariels, des exports de données et des alertes automatiques par email.

2. Architecture technique

2.1 Stack technologique

Couche	Technologie
Langage serveur	PHP 8.0+
Base de données	MySQL / MariaDB
Serveur web (dev.)	Apache (via XAMPP)
Frontend	HTML5, Bootstrap 5, Bootstrap Icons
Graphiques	Chart.js
Génération PDF	DOMPDF
Génération Excel	PhpSpreadsheet
Envoi d'emails	PHPMailer (SMTP)
Gestion des dépendances	Composer

2.2 Principe général

L'application suit une architecture procédurale simple organisée par responsabilité : configuration, authentification, fonctions utilitaires, modules métier et pages publiques. Chaque page inclut systématiquement `config/config.php`, `config/database.php` et `includes/auth.php` avant tout traitement, afin de garantir la session, la connexion base de données et les contrôles d'accès.

L'accès à la base de données passe exclusivement par PDO avec des requêtes préparées (protection contre les injections SQL).

3. Structure du projet

arborescence simplifiée du dossier actuariat_assurance/ :

```
actuariat_assurance/
├── config/
│   ├── config.php          (session, constantes, expiration 20 min)
│   ├── database.php        (connexion PDO)
│   └── mail.php             (paramètres SMTP, seuil d'alerte échéance)
├── includes/
│   ├── auth.php            (connexion, exigerRole(), exigerConnexion())
│   ├── functions.php       (helpers, moteur de règles du dashboard)
│   └── header.php / footer.php
├── modules/
│   └── polices/
│       ├── liste.php / details.php
│       ├── ajouter.php / modifier.php / supprimer.php
│       └── export_pdf.php / export_excel.php
├── public/
│   ├── accueil.php         (page de garde)
│   ├── login.php / logout.php
│   ├── dashboard.php
│   └── index.php           (point d'entrée, redirige selon la session)
├── scripts/
│   └── alerte_echeances.php (script cron, envoi d'emails)
├── sql/
│   ├── schema.sql
│   └── seed_utilisateurs.php
├── assets/                  (css, js, images)
├── composer.json
└── README.md
```

4. Modèle de données

La base actuariat_assurance comprend 7 tables InnoDB, définies dans sql/schema.sql :

Table	Rôle
utilisateurs	Comptes et rôles (admin, utilisateur_avance, standard)
branches	Branches d'assurance (Auto, Habitation, Santé, Vie, Transport)
polices	Contrats d'assurance — entité principale
sinistres	Sinistres liés à une police (paiements, réserves PSAP)
provisions_techniques	Résultats des calculs actuariels (PPNA, PRC, PSAP, PM)
traites_reassurance	Traités de réassurance par branche
journal_audit	Journal horodaté des actions sensibles

4.1 Table polices (détail)

Champ	Type	Description
numero_police	VARCHAR(30)	Identifiant unique du contrat
id_branche	INT (FK)	Référence vers la table branches
prime_pure / chargement_pct	DECIMAL	Base du calcul de la prime commerciale
prime_commerciale	DECIMAL(14,2)	Prime facturée à l'assuré
statut	ENUM	actif / suspendu / resilie / expire
alerte_echeance_envoyee	TINYINT(1)	Empêche le doublon d'email d'alerte

Relations : une **branche** a plusieurs **polices** ; une **police** a plusieurs **sinistres** (clé étrangère avec ON DELETE CASCADE) ; les tables **provisions_techniques** et **traites_reassurance** se rattachent à une branche.

5. Sécurité

5.1 Authentification et rôles

Les mots de passe sont hachés avec `password_hash()` (bcrypt) et vérifiés avec `password_verify()`. La fonction `exigerRole(array $rolesAutorises)` dans `includes/auth.php` bloque l'accès à une page si le rôle de l'utilisateur connecté n'est pas dans la liste autorisée (retour HTTP 403).

5.2 Gestion de session

Le cookie de session est configuré en `httponly` (inaccessible en JavaScript, limitant le risque XSS) et `samesite=Lax`. Une expiration automatique après 20 minutes d'inactivité déconnecte l'utilisateur et le redirige vers la page de connexion ; ce contrôle ne s'applique qu'aux utilisateurs authentifiés, afin de ne pas perturber les visiteurs anonymes sur la page de garde.

5.3 Protection CSRF

Chaque formulaire de modification (connexion, création/édition de police) intègre un jeton CSRF généré côté serveur et vérifié à la soumission, empêchant la falsification de requête intersite.

5.4 Accès aux données

Toutes les requêtes SQL utilisent des requêtes préparées PDO avec des paramètres liés, ce qui élimine le risque d'injection SQL. Les sorties HTML passent par une fonction d'échappement pour prévenir les failles XSS.

6. Fonctionnalités clés

6.1 Tableau de bord et moteur de règles

Le tableau de bord calcule six indicateurs à partir de requêtes SQL réelles (aucune donnée codée en dur) : polices actives, primes en cours, taux de sinistralité (S/P), sinistres ouverts, réserves PSAP et échéances sous 30 jours. La fonction `genererCommentaires()` (`includes/functions.php`) applique un jeu de règles à ces indicateurs pour produire des commentaires classés alerte / attention / positif, par exemple : un taux de sinistralité supérieur ou égal à 85% déclenche une alerte.

6.2 Gestion des polices (CRUD)

Le module `modules/polices/` offre la liste paginée (10 lignes/page) avec recherche et filtres, l'ajout et la modification (réservés aux rôles admin et utilisateur_avance), et la suppression (réservée au rôle admin, avec confirmation).

6.3 Exports

`export_pdf.php` génère un PDF (DOMPDF) et `export_excel.php` un fichier .xlsx (PhpSpreadsheet) de la liste des polices, en respectant les filtres de recherche actifs.

6.4 Alertes automatiques par email

Le script `scripts/alerte_echeances.php`, destiné à être exécuté quotidiennement via une tâche planifiée (cron), recherche les polices actives arrivant à échéance dans un délai configurable (7 jours par défaut) et envoie un email récapitulatif via PHPMailer. La colonne `alerte_echeance_envoyee` évite l'envoi répété de la même alerte.

7. Dépendances externes

Les bibliothèques PHP sont gérées via Composer et déclarées dans `composer.json` :

Bibliothèque	Usage
dompdf/dompdf	Génération de PDF (export polices)
phpoffice/phpspreadsheet	Génération de fichiers Excel
phpmailer/phpmailer	Envoi d'emails SMTP (alertes d'échéance)

Le dossier `vendor/` généré par Composer n'est volontairement pas versionné dans Git (présent dans `.gitignore`), car il est régénérable via `composer install`.

8. Installation et déploiement

Environnement local (XAMPP)

- 1 Placer le dossier du projet dans htdocs/ (Apache et MySQL démarrés) ;
- 2 Importer sql/schema.sql via phpMyAdmin (crée la base et les tables) ;
- 3 Exécuter sql/seed_utilisateurs.php dans le navigateur (crée les 3 comptes de test avec mots de passe hachés) ;
- 4 Exécuter composer install à la racine du projet (installe DOMPDF, PhpSpreadsheet, PHPMailer) ;
- 5 Configurer les identifiants SMTP dans config/mail.php ;
- 6 Accéder à public/index.php pour démarrer l'application.

Les mots de passe hachés ne sont pas insérés en dur dans schema.sql : le hachage bcrypt doit être généré par PHP (password_hash) pour être garanti compatible avec la vérification, d'où l'exécution séparée de seed_utilisateurs.php.

9. Conventions de code

9.1 Langue et nommage

L'ensemble du code (variables, fonctions, noms de tables et de champs, commentaires) est rédigé en **français**, choix cohérent avec le contexte métier du projet. Les conventions de nommage suivent le standard PHP courant :

Élément	Convention	Exemple
Variables et fonctions	camelCase	\$primeTotale, exigerRole()
Tables et champs SQL	snake_case	date_expiration, id_police
Constantes	MAJUSCULES_SOUS_TIRETS	DUREE_MAX_INACTIVITE
Fichiers PHP	minuscules_avec_underscore	seed_utilisateurs.php

9.2 Organisation du code

Chaque page suit systématiquement le même ordre d'inclusion : configuration, connexion base de données, authentification, fonctions utilitaires, puis logique métier de la page, et enfin affichage (header / contenu / footer). Cette convention facilite la lecture et la maintenance : un développeur reprenant le projet retrouve la même structure sur n'importe quelle page.

Exemple d'en-tête standard d'une page du module polices :

```
require_once __DIR__ . '/../../config/config.php';
require_once __DIR__ . '/../../config/database.php';
require_once __DIR__ . '/../../includes/auth.php';
require_once __DIR__ . '/../../includes/functions.php';

exigerRole(['admin', 'utilisateur_avance']);
```

9.3 Sécurité et validation

- Toute requête SQL utilise des paramètres liés PDO, jamais de concaténation directe de variables ;
- Toute donnée affichée dans le HTML passe par une fonction d'échappement avant impression ;
- Tout contrôle d'accès à une page passe par `exigerConnexion()` ou `exigerRole()`, appelé en tout début de fichier ;
- Les identifiants sensibles (SMTP) sont isolés dans `config/mail.php`, séparé du reste de la configuration.

9.4 Commentaires

Chaque fonction non triviale est précédée d'un bloc de commentaire expliquant son rôle. Les sections de fichier SQL et PHP sont séparées par des blocs de commentaires numérotés, facilitant la navigation dans les fichiers longs comme `schema.sql` ou `dashboard.php`.

10. Pistes d'évolution

- Étendre le CRUD complet aux sinistres et aux provisions techniques ;
- Ajouter une page de gestion des comptes utilisateurs pour l'administrateur ;
- Historiser les modifications de polices (traçabilité fine au-delà du journal d'audit) ;
- Ajouter des tests automatisés (PHPUnit) sur les fonctions de calcul du dashboard ;
- Passer le mot de passe SMTP en variable d'environnement plutôt qu'en dur dans config/mail.php.